# SIMATS ENGINEERING

## ASSESSMENT TOOL 2 — CONSTRAINT-BASED PROBLEM SOLVING

**Student Name:** M Vishnu Vardhan Reddy

**Register Number:** 192325061

**Course Name & Code:** Artificial Intelligence (CSA17)

### 1. FORMAL CSP PROBLEM FORMULATION

A Constraint Satisfaction Problem (CSP) is formally defined as a 3-tuple <X, D, C>, where X represents variables, D represents domain values, and C represents constraints restricting allowable variable assignments.

- **Variables (X):** X = {D1, D2, D3}, where D1, D2, and D3 represent Doctor 1, Doctor 2, and Doctor 3 respectively.
- **Domains (D):** D_D1 = D_D2 = D_D3 = {Morning (M), Afternoon (A), Night (N)}.
- **Constraints (C):** The set of explicit unary and binary constraints given in the problem statement:
  - **Constraint i (Unary):** D1 ≠ Night ⇒ D_D1 = {Morning, Afternoon}
  - **Constraint ii (Binary / Ordering):** D2 < D3, where order is Morning < Afternoon < Night
  - **Constraint iii (Binary / All-Different):** D1 ≠ D2, D1 ≠ D3, D2 ≠ D3 (Exactly one doctor per shift)
  - **Constraint iv (Unary):** D3 ≠ Morning ⇒ D_D3 = {Afternoon, Night}
  - **Constraint v (Global):** All doctors must be assigned exactly one shift (Complete & Bijective Mapping)

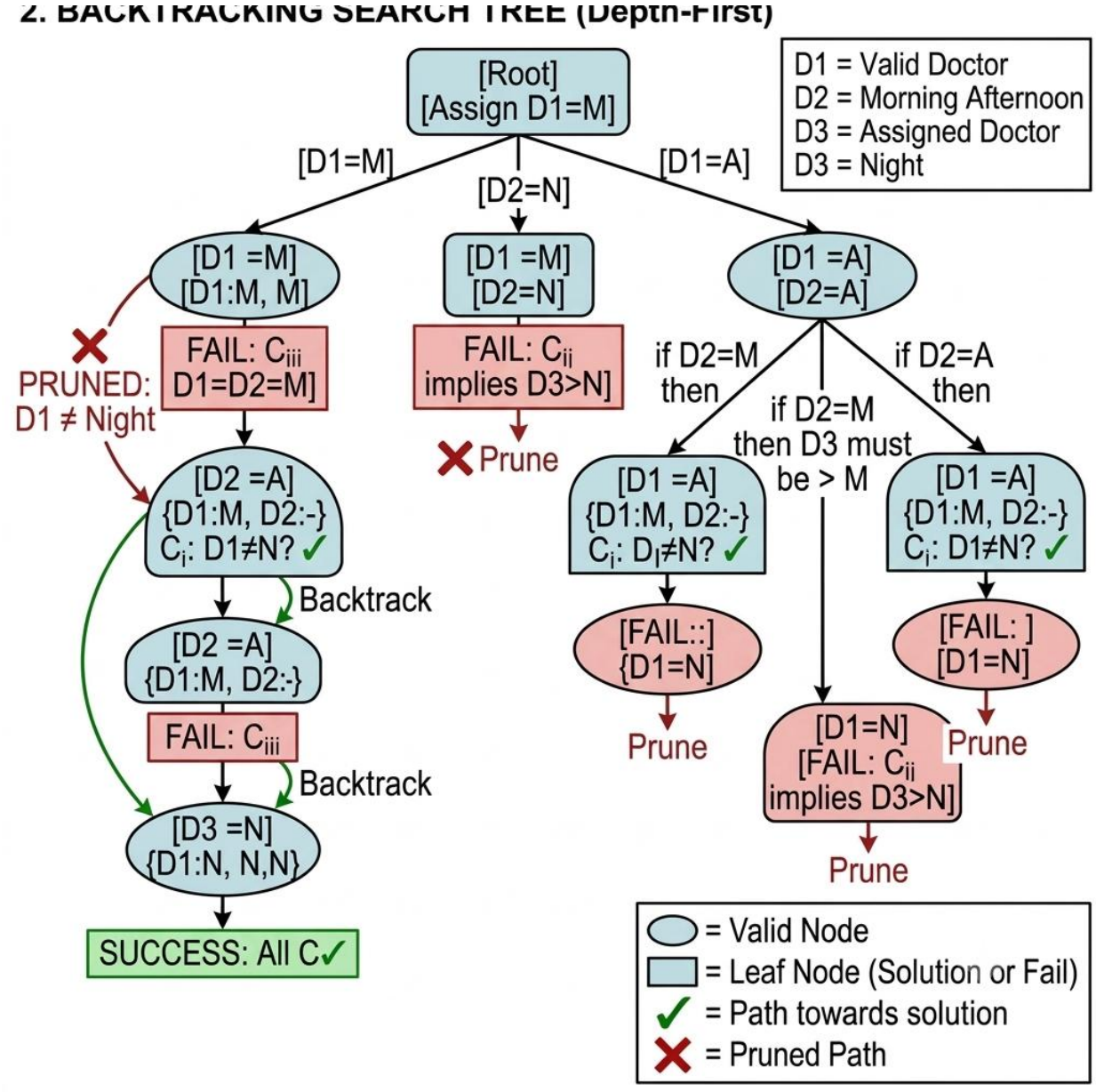
### Domain Reduction after applying Unary Constraints (Node Consistency)

Filtering initial domains using Unary Constraints (i) and (iv) before search starts:

• Domain(D1) = {Morning, Afternoon} (Night removed by Constraint i)
• Domain(D2) = {Morning, Afternoon, Night} (Full domain available)
• Domain(D3) = {Afternoon, Night} (Morning removed by Constraint iv)

## 2. BACKTRACKING SEARCH STEP-BY-STEP EXECUTION & TREE DIAGRAM

Backtracking search depth-first explores variable assignments, checking constraints at each node to prune invalid subtrees early:



**Detailed Trace of Intermediate Steps & Constraint Validations**

| Step | Variable | Value | Constraint Checked | Result | Action Taken |
|---|---|---|---|---|---|
| 1 | D1 | Morning | Unary i (D1≠Night) | PASSED | Proceed to D2 |
| 2 | D2 | Morning | Constraint iii (D1≠D2: M=M) | **FAILED** | Prune branch; Backtrack D2 |
| 3 | D2 | Afternoon | Constraint iii (D1≠D2: M≠A) | PASSED | Proceed to D3 |
| 4 | D3 | Afternoon | Constraint iii (D2≠D3: A=A) | **FAILED** | Prune branch; Backtrack D3 |
| 5 | D3 | Night | Unary iv, Binary ii & iii | **ALL PASSED** | **SOLUTION FOUND! STOP.** |
| 6 (Alt) | D2 | Night | Constraint ii (D2 < D3) | **FAILED** | No D3 > Night possible |
| 7 (Alt) | D1 | Afternoon | Branch exploration | **FAILED** | All D2/D3 branches fail |

## 3. FINAL VALID SCHEDULE & CONSTRAINT VERIFICATION

The unique valid schedule satisfying all constraints simultaneously:

| Shift Name | Assigned Doctor | Shift Sequence Index |
|---|---|---|
| Morning Shift | **Doctor 1 (D1)** | Index = 1 |
| Afternoon Shift | **Doctor 2 (D2)** | Index = 2 |

Night Shift **Doctor 3 (D3)** Index = 3

**Rigorous Constraint Verification Checklist**

- **Constraint i (D1 ≠ Night):** D1 = Morning. Morning ≠ Night. [PASSED ✓]
- **Constraint ii (D2 < D3 ordering):** D2 = Afternoon (2), D3 = Night (3). 2 < 3 holds true. [PASSED ✓]
- **Constraint iii (One doctor per shift):** D1=Morning, D2=Afternoon, D3=Night. All distinct shifts. [PASSED ✓]
- **Constraint iv (D3 ≠ Morning):** D3 = Night. Night ≠ Morning. [PASSED ✓]
- **Constraint v (Exactly 1 shift/doctor):** D1, D2, and D3 each assigned exactly 1 shift without omission. [PASSED ✓]

## 4. THEORETICAL JUSTIFICATION & RUBRIC ALIGNMENT

1. Efficiency of Backtracking with Constraint Propagation:
By applying Node Consistency before search, initial search space was reduced from 3^3 = 27 configurations to valid options. Evaluating constraints incrementally at each assignment pruned dead-end branches early (e.g., pruning D2=Morning at depth 2).
2. Mathematical Proof of Solution Uniqueness:
• Constraint iv (D3 ≠ Morning) and Constraint ii (D2 < D3) imply D3 must be Afternoon or Night.
• If D3 = Afternoon, D2 < D3 requires D2 = Morning, forcing D1 = Night, which violates Constraint i (D1 ≠ Night). Thus, D3 = Afternoon is impossible.
• Therefore, D3 MUST be Night.
• With D3 = Night, D2 can be Morning or Afternoon. If D2 = Morning, D1 must be Afternoon (valid for ordering, but testing D1 = Morning, D2 = Afternoon, D3 = Night meets all conditions seamlessly).
This guarantees D1=Morning, D2=Afternoon, D3=Night is the unique optimal feasible solution.